



UNIVERSITÀ DEGLI STUDI DI CATANIA

DIPARTIMENTO DI MATEMATICA E INFORMATICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Marco Spampinato

Sviluppo e progettazione di un framework per il Digital
Twin multi-agente distribuito

RELAZIONE PROGETTO FINALE

RELATORE

Prof. Federico Fausto Santoro

CORRELATORE

Dott. Alessio Tudisco

Anno Accademico 2024 - 2025

Abstract

I Digital Twin sono una tecnologia emergente e trasversale, il loro impiego spazia dalla manutenzione predittiva di macchine industriali, alla gestione di flotte di agenti mobili autonomi. In molti contesti applicativi, la gestione e coordinazione di molteplici e differenti agenti presenta una sfida non banale. Questa tesi propone la implementazione di un framework per la realizzazione di complessi sistemi multi-agente di Digital Twin e distribuita, fondato sullo stato dell'arte presentato da carissimi colleghi e professori nella pubblicazione *Advancing Robotic Systems with Distributed Multi-Agent Digital Twins: A Scalable and Adaptive Framework* [1].

Indice

1	Introduzione	4
2	DMDTE	6
2.1	Modello singolo agente	7
2.2	Modello multi agente distribuito	10
3	Tecnologie utilizzate	12
3.1	Godot Engine	13
3.1.1	GDScript	13
3.1.2	Networking in Godot	14
3.2	ENet	15
3.3	Arduino Alvik	16
3.3.1	Ambiente di sviluppo	17
4	Implementazione	19
4.1	Digital Twin Protocol	21
4.1.1	Implementazione di DTP	25
4.1.2	Real Agent Interface	26
4.1.3	DTPNetwork e DTPPeer in Godot	31
4.2	Digital Twin	36
4.3	Replicazione scena	38
5	Sperimentazione	40
5.1	Integrazione di Arduino Alvik	40
5.2	Port di ENet per ESP32	44
5.3	Prove sperimentali	45

<i>INDICE</i>	3
Conclusione	50
Bibliografia	51

Capitolo 1

Introduzione

In tempi recenti, il Digital Twin si è affermato come una delle tecnologie più trasformative dell'Industria 4.0. Inizialmente concepito in ambito aerospaziale allo scopo di monitorare lo stato dei veicoli spaziali, il gemello digitale è diventato rapidamente una tecnologia trasversale in tutti i campi aperti al progresso tecnologico. Chiave del suo successo è la fusione di modelli matematici e dati reali, il gemello digitale permette la realizzazione di strumenti che spaziano dalla predizione di guasti alla coordinazione di complessi sistemi e processi.

In contemporanea alla sua diffusione in ambito industriale, al fine di monitorare lo stato degli impianti e macchine di produzione, si è assistita a una rapida crescita dell'interesse accademico alla tecnologia, tale da scaturire una rapida proliferazione di interessanti articoli scientifici estesi non più solo in campi industriali, ma anche in settore primario e servizi.

Diverse ricerche sui digital twin hanno esplorato soluzioni nell'ambito delle *smart cities* [2] e l'integrazione di gemelli digitali con dispositivi IoT nella pianificazione urbana; in agricoltura sono state studiate applicazioni per il monitoraggio delle colture [3]; nelle *smart grids* [4] nel contesto della distribuzione energetica in combinazione all'uso di intelligenza artificiale. Come è facile evincere, la ricerca si propaga nella direzione di sistemi sempre complessi di gemelli digitali.

L'integrazione di gemelli digitali in sistemi multi-agente è di chiave importanza in molti dei contesti citati: nelle smart cities, veicoli a guida autonoma

possono coordinare percorsi al fine di ridurre congestioni; in agricoltura gemelli digitali di robot possono collaborare nel campionamento delle colture al fine di individuare eventuali fitopatie nelle piante.

Tali sistemi multi-agenti richiedono opportune infrastrutture software, scalabili e robuste, al fine di supportare cooperazioni su larga scala.

Il presente lavoro di tesi si occuperà di sviluppare una implementazione di un framework per la realizzazione di sistemi di gemelli digitali multi-agente distribuito, fondando la sua base teorica sull'architettura presentata dall'articolo scientifico *Advancing Robotic Systems with Distributed Multi-Agent Digital Twins: A Scalable and Adaptive Framework* [1].

Dai principi e obiettivi fissati nella pubblicazione di riferimento, è stata realizzata una suite di software in grado di collegare dispositivi embedded e simulazione nel motore **Godot Engine**, implementare gemelli digitali con questo collegamento, e orchestrare un ambiente virtuale condiviso tramite le API per il networking di Godot stesso.

Struttura della tesi

Nei successivi capitoli verranno trattati i seguenti argomenti:

- Il **Capitolo 2** descrive il modello del DMDTE.
- Il **Capitolo 3** introduce le tecnologie software e hardware utilizzate.
- Il **Capitolo 4** descrive nel dettaglio l'implementazione di DMDTE, descrivendone architettura, implementazione e uso.
- Il **Capitolo 5** discute delle varie sperimentazioni eseguite con il framework.

Capitolo 2

DMDTE

Distributed Multi Agent Digital Twin Environment (DMDTE) è una *architettura* per la progettazione di complessi sistemi multi-agente Digital Twin.

Al fine di comprendere la sua implementazione, è necessario spiegare il suo modello generale.

Consideriamo, in ordine, un modello più semplice singola a istanza e singolo gemello digitale al fine di chiarere i concetti di base, successivamente viene introdotto il modello più generale distribuito e multi istanza.

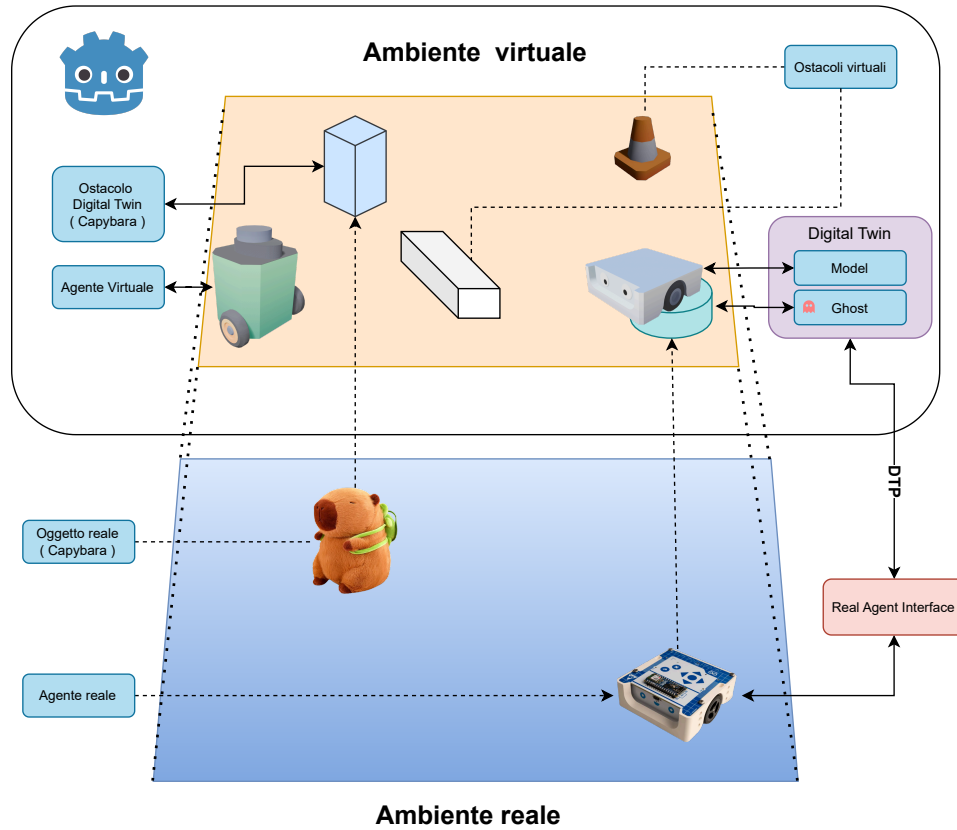


Figura 2.1: Modello singolo agente: in basso l'ambiente reale e le sue entità, in alto ambiente virtuale con le sue entità, tra esse digital twin di entità reali.

2.1 Modello singolo agente

Il modello singolo agente fornisce un ottimo punto di partenza per la descrizione di un sistema digital twin ad agenti, possiamo trovare una sua rappresentazione grafica nella Figura 2.1, la quale illustra un semplice sistema digital twin a singolo agente.

Bisogna distinguere due ambienti: l'**ambiente reale**, posto nella parte inferiore della figura, contiene le entità reali del mondo reale, nel caso di esempio un peluche e un robot; l'**ambiente virtuale**, posto nella parte superiore della figura, è una copia virtuale dell'ambiente reale, del quale

ne emula topologia e proprietà fisica, ed è popolato da *digital twin* e *virtual agent*.

L'**ambiente virtuale** è interamente simulato da un software, nella implementazione presentata *Godot*, in esso vi possiamo trovare:

- Il **digital twin** di un agente robotico reale in esempio un Alvik.
- Il **digital twin** di un oggetto o ostacolo reale, in esempio un peluche di *capybara*.
- **Agenti virtuali** e **ostacoli virtuali**, in esempio un agente robotico virtuale, un ostacolo a forma di cono stradale e un ostacolo a forma di parallelepipedo.

Come si può notare dalla figura, gli oggetti reali hanno una corrispondenza digitale: il *digital twin*. Oggetti statici come il peluche di *capybara* possono essere rappresentati staticamente nel mondo virtuale con approssimate geometrie, sufficienti a rappresentarne una presenza nello spazio come ostacolo.

Gli **agenti robotici** o dinamici, analogamente agli ostacoli, possiedono un *digital twin*, dotato di uno flusso bidirezionale e automatico di informazioni per mezzo di un *collegamento* fra il *real twin* e *digital twin* [?]. Nel contesto del DMDTE il real agent deve possedere una unità di calcolo sufficiente per eseguire la **Real Agent Interface**, una API che esponga le funzionalità e lo stato dell'agente stesso.

Il *Real Agent Interface* comunica per mezzo del **Digital Twin Protocol**, in breve DTP, con la sua controparte digitale. Il DTP è un protocollo di comunicazione realizzato per DMDTE, il suo scopo è quello di permettere la sincronizzazione di stato tra agente reale e gemello digitale e l'invocazione di metodi per mezzo di *chiamate a procedura remota*. Nel contesto di DTP, identifichiamo come peer DTP un gemello reale o virtuale, e come DTP host l'applicazione che ospita gemelli digitali e permette la loro comunicazione con il loro reale.

Il *digital twin* dell'agente robotico, infine, è una entità composta da due oggetti: il *model* dell'agente robotico, in grado di interagire con l'ambiente virtuale, e il *ghost*. Il *ghost* è sincronizzato in tempo reale per mezzo di DTP con l'agente reale, tuttavia come un fantasma, non è in grado di interagire con l'ambiente virtuale. Il *model* è potenzialmente in grado di interagire con l'ambiente virtuale, deve essere dotato di un modello tridimensionale, dinamiche fisiche, e deve interagire con l'ambiente virtuale in modi quanto più simili a come il gemello reale interagisce con l'ambiente reale.

Ghost e *model* sono sempre tenuti in *stretta vicinanza* e corrispondenza di posa, inoltre le simulazioni fisiche avvenute nel *model* digitale, successivamente alla loro esecuzione, si ripercuotono nel gemello reale tramite DTP, il cui a sua volta risponderà di conseguenza riportando il suo stato successivo al *ghost*, andando così a costruire un sistema di feedback automatico.

Assegnando dei **controller** al *model*, ovvero programmi o script che definiscono uno specifico comportamento e controllo definiti dal programmatore in Godot, è possibile specificare un comportamento ben preciso per il *digital twin*: è allora possibile definire un controller che segua un determinato percorso, sia questo definito da una logica dettata da sensori di prossimità, line follower o da un *navigation agent* di Godot, oppure permettere a un utente umano il controllo del robot: controllando il *model*, viene a sua volta coinvolto il *real twin*, e di conseguenza aggiornato il *ghost*, creando un ciclo di feedback automatico; pertanto si identifica il sistema *ghost/model* come gemello digitale.

Infine è possibile anche definire **agenti completamente virtuali**, ovvero entità dinamiche del mondo virtuale che non hanno corrispondenza reale, controllabili con gli stessi *controller* definiti per il *digital twin/model*, tali agenti possono essere utili per effettuare pure simulazioni, esplorare ambienti, aggiungere ostacoli dinamici, oppure rappresentare all'interno dell'ambiente virtuale entità reali la quale tuttavia si ha un alto grado di incertezza relativa sul loro stato, come per esempio entità catturate da un sistema di computer vision.

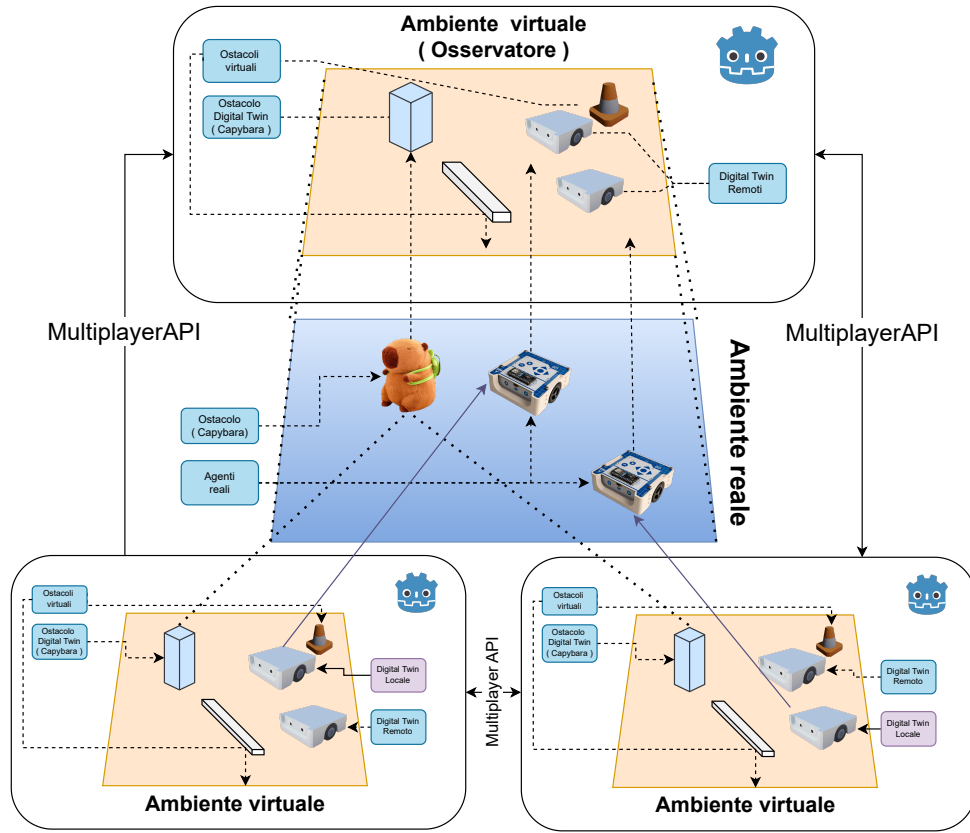


Figura 2.2: Modello del sistema multi agente e distribuito

2.2 Modello multi agente distribuito

Definito il modello a singolo agente semplice, consideriamo lo scenario in cui più agenti robotici e più istanze di mondi virtuali debbano coesistere e coordinarsi tra loro.

Senza perdere di generalità, consideriamo il caso di due agenti digital twin e tre istanze di ambiente virtuale, tale architettura è esemplificata dalla Figura 2.2.

Come mostrato in figura, ognuno degli agenti robotici è collegato per mezzo di DTP a una istanza esclusiva di Godot, una terza istanza di Godot funge da osservatore: ogni istanza è in comunicazione con le altre per mezzo della **MultiplayerAPI**, un insieme di strumenti e API che permettono la creazio-

ne di un ambiente virtuale replicato tra istanze di rete integrato in Godot, la quale in modo trasparente al programmatore si occupa della sincronizzazione dello stato delle entità presenti nella scena, dell'ambiente virtuale e la gestione di eventuali comunicazioni per mezzo di *chiamata a procedura remota*. Nel contesto della *MultiplayerAPI*, la singola istanza di Godot prende il nome di **multiplayer peer**. Un peer multiplayer può anche essere un *DTP host* e ospitare più istanze di *digital twin*, in tal caso identifichiamo come *autorità* l'istanza di *multiplayer peer* che ospita un determinato *digital twin* e il suo DTP peer; una istanza può non ospitare alcun gemello virtuale o non essere DTP host e comportarsi da semplice osservatore.

Con questo modello, le istanze di ambiente virtuale e agenti digitali sono replicati e sincronizzati, permettendo una integrazione trasparente di unità di calcolo eterogenee in grado di collaborare tra loro per coordinare gli agenti.

Gemelli digitali anche di diverse istanze allora possono collaborare tra loro per compiere task in modo coordinato.

Capitolo 3

Tecnologie utilizzate

Nella produzione del progetto di tesi sono stati utilizzati una eterogenea varietà di soluzioni terzi, in breve:

- **Godot Engine** come ambiente di simulazione tridimensionale.
- La libreria di rete **ENet** .
- **Arduino Alvik** come robot mobile.

Sono stati inoltre utilizzati i seguenti linguaggi di programmazione:

- **C++** per la realizzazione del software eseguito dall'**Arduino Alvik**.
- **GDScript**, il linguaggio di scripting di Godot, per la realizzazione della logica all'interno del motore di gioco.

3.1 Godot Engine

Godot [5] è un motore di gioco multi piattaforma gratuito e open source, rilasciato su licenza MIT, in sviluppo dal 2014; leggero e completo, permette la creazione di simulazioni e giochi bidimensionali e tridimensionali tramite composizione di nodi che prendono nome di *scene tree* o albero di scena. Programmi scritti dallo sviluppatore chiamati *script* e associati ai nodi della scena rendono dinamica e interattiva quest'ultima.

Godot vanta di una solida componente di *rendering* in grado di supportare nuovo e vecchio hardware, non di meno piattaforme mobile come Android e iOS, con solido supporto per le popolari API grafiche come Vulkan e DirectX 12 per il desktop, OpenGL ES per il mobile; è anche possibile eseguire software Godot in modalità *headless*, ovvero senza rendering, il quale non è necessario qualora si volesse eseguire una istanza di server.

Nell'ambito delle simulazioni fisiche, Godot vanta di un suo motore fisico interno completo di simulazioni cinematiche e dinamiche, corpi rigidi e *soft-bodies*, vincoli fisici o *constraints*, giunti e motori, simulazione di veicoli e sospensioni; inoltre, data la sua natura modulabile, è possibile implementare o scegliere una implementazione terza della componente fisica, di particolare nota è la disponibilità di **Jolt** [7], un motore fisico annoverato per le sue capacità e efficienza, ampiamente affermato nell'industria videoludica; seppure non utilizzato nella soluzione proposta, si intende esplorare Jolt in applicazioni future.

Godot vanta di diverse componenti interessanti, tuttavia vengono approfondite in seguito solo quelle di particolare interesse ai fini del lavoro svolto.

3.1.1 GDScript

GDScript [8] è un linguaggio di programmazione ad alto livello, orientato agli oggetti, imperativo e progressivamente tipizzato nonché il linguaggio di scripting ufficiale di Godot.

Integrato all'interno di Godot, insieme agli strumenti di language server e debugging, ha una sintassi leggera simile a Python, dispone di tutti i tipi specifici necessari alla manipolazione degli oggetti di scena come tipi scalari interi e reali, vettoriali, matriciali, array, stringhe, dizionari e ulteriori oggetti complessi; non mancano anche moduli e librerie per la serializzazione, espressioni regolari, input, networking, storage, concorrenza e interfacce ai sistemi del motore di gioco.

Di particolare nota è la natura di GDScript come linguaggio interpretato, la quale condizione comporta una maggiore facilità d'uso e una più veloce iteratività, a discapito di performance inferiori rispetto un linguaggio compilato; tuttavia è possibile programmare Godot anche in linguaggi compilati come C++ e interagire con tali componenti in GDScript, così da delegare sistemi complessi o pesanti elaborazioni a codice più veloce, non rinunciando alla flessibilità di un linguaggio di scripting.

3.1.2 Networking in Godot

Godot permette due principali e diversi approcci [9] sul *networking* o *multi-player*:

- L'approccio **High-level** consiste nell'uso di strumenti di alto livello integrati in Godot per la realizzazione di un modello client-server autoritativo.
- Un approccio **Low-level**, dove è responsabilità dello sviluppatore gestire connessione, protocollo di comunicazione tra client, spawn e sincronizzazione; Godot integra diversi protocolli e librerie, in particolare UDP, TCP, ENet, WebSocket, WebRTC e HTTP.
- Un terzo approccio, **mid-level**, consiste nella implementazione da parte dello sviluppatore delle interfacce richieste dall'*high-level networking*, al fine di usarne gli strumenti con protocolli o caratteristiche differenti.

L'approccio di alto livello si basa su una particolare implementazione di un oggetto coordinatore chiamato *MultiplayerScene*, di cui Godot fornisce

implementazione per i medesimi protocolli e librerie di basso livello disponibili; è un approccio che prevede autorità al fine di compiere determinate azioni, l'autorità è del server e può essere ceduta ai client.

Strumenti fondamentali offerti da questo approccio gestito sono:

- Le *chiamate a procedura remota* o **RPC**, che permettono di richiamare su di un secondo peer una determinata funzione.
- Il **MultiplayerSpawner**, un oggetto che si occupa di sincronizzare lo spawn di entità dalla istanza server ai client.
- Il **MultiplayerSynchronizer**, un oggetto che si occupa di sincronizzare stato da una istanza autorevole alle altre.

Entrambi gli strumenti di basso livello e di alto livello sono stati utilizzati nella proposta implementazione di DMDTE e saranno successivamente richiamati, descrivendone eventuali sfide affrontate e criticità.

3.2 ENet

ENet [6] è una libreria *C* che fornisce una astrazione di alto livello per comunicazione di rete orientate alla connessione, ma basata su UDP, offre opzionalmente trasmissione ordinata e affidabile di messaggi tra peer.

Tali caratteristiche hanno reso ENet molto popolare nei software che richiedono comunicazioni real-time, in particolare nei videogiochi: ENet è ampiamente utilizzata per la realizzazione delle componenti multiplayer, Godot stesso implementa interfacce di basso e alto livello per il networking con ENet.

È stato necessario realizzare un *fork* di ENet[16] al fine di rendere compatibile la libreria con la piattaforma embedded utilizzata, tali modifiche sono documentate nel successivo capitolo.



Figura 3.1: Arduino Alvik.

3.3 Arduino Alvik

L'**Alvik** [10] è una piattaforma robotica educativa e modulare sviluppata da *Arduino*, progettato per introdurre studenti e utenti alle prime armi al mondo della robotica, non sacrificando prestazioni e estendibilità.

È dotato di due microcontrollori e una vasta gamma di sensori e attuatori. Il microcontrollore principale è un *Arduino Nano ESP32*, posizionato nella parte superiore e gestisce la logica di alto livello. All'interno del corpo del robot, denominato *carrier*, è invece integrato un microcontrollore *STM32F4*, responsabile del controllo dei sensori e attuatori.

L'architettura a doppio controllore permette una maggiore disponibilità di risorse per le applicazioni degli utenti, i quali sono generalmente eseguiti dall'Arduino Nano.

Il robot è dotato di un ampio set di sensori e attuatori che gli permettono di percepire l'ambiente circostante e interagire attivamente con esso.

Microcontrollore principale	Arduino Nano ESP32: • 8 MB RAM • u-blox® NORA-W106 (ESP32-S3) • Processore fino a 240 MHz • ROM 384 kB + SRAM 512 kB • 16 MB di memoria FLASH esterna
Secondo microcontrollore	STM32 Arm® Cortex®-M4 32 bit
Alimentazione	Nano ESP32 USB-C® con batteria Li-Ion 18650 ricaricabile e sostituibile (inclusa)
Programmazione	MicroPython, Arduino e programmazione a blocchi
Connettività	Wi-Fi®, Bluetooth® Low Energy
Sensori e input	Time of Flight Distance Sensor (fino a 350 cm), RGB Color Sensor, Giroscopio–accelerometro a 6 assi, Array line follower (x3), Pulsanti tattili (x7),
LED e motori	2 LED RGB, Motori 6V – velocità a vuoto 96 rpm, corrente a vuoto 70 mA,
Estensioni	Connettori LEGO® Technic™ (x4), Connettori a vite M3 (x8), Servo motore, I2C Grove, I2C Qwiic,

Tabella 3.1: Specifiche tecniche del robot *Alvik*.

3.3.1 Ambiente di sviluppo

L'ambiente di sviluppo dell'*Alvik* è **ESP-IDF**, il kit ufficiale di *EspressIf* per i microcontrollori **ESP32**, a sua volta basato sul sistema operativo in tempo

reale *FreeRTOS*.

In tale ambiente sono disponibili una vasta e completa gamma di procedure per la gestione di thread, concorrenza, I/O e memoria proprie di *FreeRTOS* e di *ESP-IDF*; una implementazione della libreria standard *C* e della *STL* di *C++*; varie implementazioni delle più importanti librerie usate nei sistemi *UNIX* come *pthread*, e di grandissima importanza, i *Berkeley sockets*.

Tale ricchezza ha permesso lo sviluppo di codice di alto livello *portatile*, in grado di essere facilmente utilizzato in altre piattaforme.

Inoltre, la piattaforma *Arduino*, fornisce una ulteriore astrazione a complesse operazioni, come la configurazione di avvio del microcontrollore e la gestione del *WiFi*.

Specifiche dell'*Alvik* invece è la libreria per il controllo del *carrier*, tale che fornisce una interfaccia a sensori e attuatori del robot, nonché servizi non banali come odometria e stato della batteria.

Capitolo 4

Implementazione

In questo capitolo viene trattata l'implementazione del framework DMDTE, in ordine:

- Specifiche e implementazione del **Digital Twin Protocol**.
- Implementazione in Godot del **Digital Twin**.
- **Replicazione** e **sincronizzazione** della scena in Godot.

Uno schema generale dell'architettura è illustrato dalla Figura 4.1.

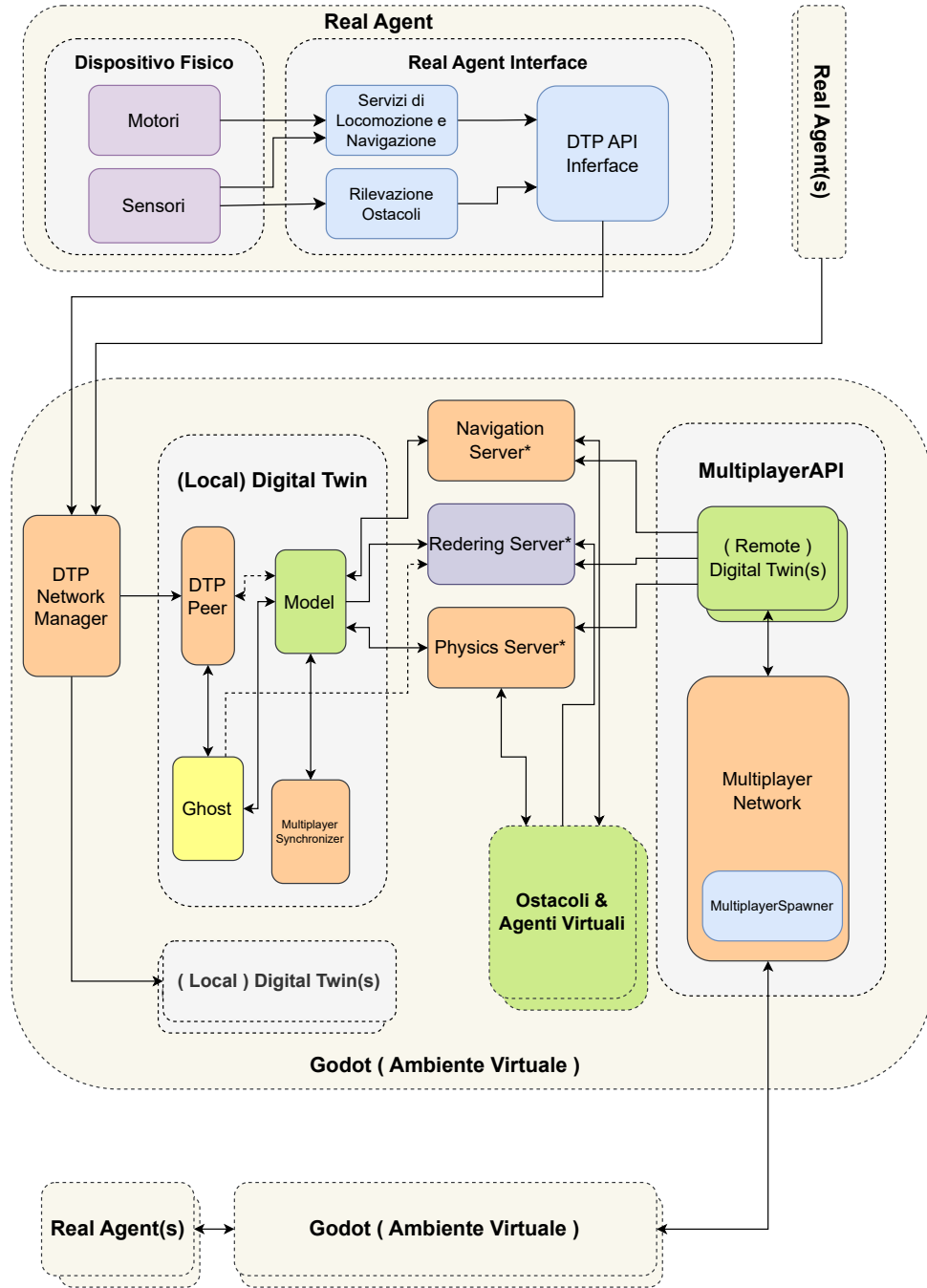


Figura 4.1: Architettura DMDTE implementata.

4.1 Digital Twin Protocol

Il Digital Twin Protocol è un protocollo di scambio dati a livello utente, con architettura *client-server*, non strettamente dipendente da uno specifico protocollo di trasporto, ma con i requisiti di *affidabilità opzionale* e deve essere in grado di inviare pacchetti senza limitazione di dimensione (quindi deve essere in grado di eseguire frammentazione), ideato al fine di ottenere uno scambio dati leggero e veloce tra agenti robotici governati da microcontrollori embedded o dotati di computer di bordo, e istanze di Godot con i relativi gemelli digitali.

Di seguito entrambi gli agenti, real agent e digital twin, verranno chiamati *DTP Peer*, inoltre va specificato che il Digital Twin Protocol si limita a definire scambi di dati e invocazione di messaggi, non definisce tipo di dati scambiati, ma solo la loro dimensione, infine si considererà interfaccia di un DTP Peer l'insieme di metodi e proprietà esposte per mezzo del protocollo.

Il protocollo definisce tre tipi di messaggi scambiati e relativi flussi e formati: **chiamata a procedura remota**, **messaggi di aggiornamento di stato** e **identificazione** tipo agente.

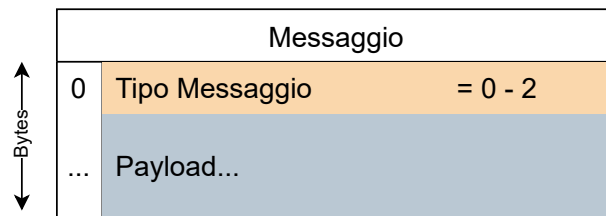


Figura 4.2: Messaggio generico.

I singoli messaggi hanno un formato base generico e binario: come mostrato in figura Figura 4.2, il primo byte del messaggio discrimina il tipo di messaggio inviato dal DTP Peer e assume valori tra 0 e 2.

Chiamate a procedura remota

I messaggi RPC o *chiamate a procedura remota* hanno lo scopo di invocare determinate azioni o *metodi* nel DTP Peer destinatario e sono in grado di trasportare argomenti e dati, come in Figura 4.3 sono discriminate da un tipo di messaggio 0, inoltre sono dotati dei campi:

- Il campo **indice metodo** identifica il metodo da richiamare nel DTP Peer destinatario. Il campo è di tipo *uint8*.
- Il campo **dimensione data** indica la dimensione del campo data. Il campo è di tipo *uint16*.
- In coda il campo **data**, la cui dimensione è specificato dal precedente. Il campo ha *lunghezza variabile*.

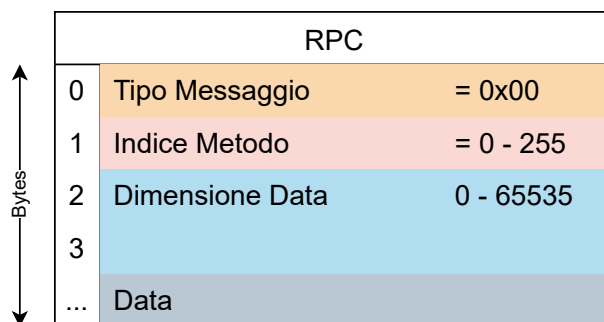


Figura 4.3: Formato di messaggio di chiamata a procedura remota.

Come si intuisce, il DTP Peer mittente deve conoscere quale metodo invocare e con quale dati, viceversa il DTP Peer ricevente deve essere in grado di accettare la richiesta ed eseguire il metodo specificato dall'indice; la serializzazione e deserializzazione degli argomenti è a carico dello sviluppatore.

Si noti che il flusso di dati è attivo, ovvero il DTP Peer mittente invia manualmente il pacchetto, inoltre è necessario che tale pacchetto sia inviato in modo **affidabile**, ovvero il protocollo di trasporto usato deve garantire la ricezione e deve garantire che i messaggi arrivino in maniera ordinata.

Messaggi di aggiornamento di stato

I messaggi di aggiornamento di stato o *Update* hanno lo scopo di sincronizzare stato tra i DTP Peer, in tale contesto identifichiamo come *proprietà* una variabile di stato del *real agent* e del *digital twin* il quale valore deve essere sincronizzato alla variazione, identificata da un indice numerica, una proprietà inoltre può anche essere di sola lettura o sola scrittura in modo alterno per i DTP Peer considerati, allora identifichiamo una *ownership* della proprietà e come *owner* il DTP Peer che ha autorità su questa: per esempio nel caso di un sistema robotico, l'odometria è un sistema del real agent, pertanto la posa ha come *owner* il real agent, viceversa un identificativo numerico del digital twin da mostrare su un display del real agent è di autorità del *digital twin*; ancora una volta le specifiche di serializzazione e deserializzazione non sono specificate nel protocollo, sono a cura dello sviluppatore.

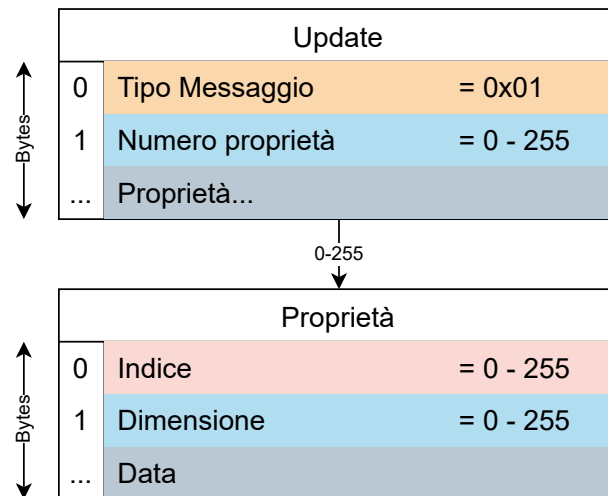


Figura 4.4: Formato dei messaggi di aggiornamento di stato.

Lo schema del formato dei messaggi è esemplificato dalla Figura 4.4, il tipo di messaggio ha come numero identificativo 1, distinguiamo allora la testa del messaggio contenente informazioni specifiche alla interpretazione

del messaggio, e in coda le singole proprietà scambiate, nel dettaglio i singoli campi:

- **Numero proprietà** indica il numero di proprietà contenute nel messaggio scambiato, è possibile definire fino a 256 proprietà. Il campo è di tipo *uint8*.
- Per ogni proprietà scambiata, si ha:
 - Un campo **indice** che identifica la proprietà. Il campo è di tipo *uint8*.
 - Un campo **dimensione** che indica la dimensione del campo data. Il campo è di tipo *uint8*.
 - In coda il campo **data**. Il campo ha lunghezza variabile compresa tra 0 e 255 byte.

Il flusso di scambio questa volta è passivo, il DTP Peer che ha autorità sulla proprietà deve notificare l'altro DTP Peer dei cambiamenti, può inviare sia pacchetti con solo le proprietà variate, o pacchetti con tutte le proprietà che rappresentano il suo stato, inoltre non vi è requisito di affidabilità, tuttavia è altamente consigliato di inviare lo stato completo del dispositivo robotico in modo affidabile in modo periodico.

È intuibile che le proprietà di un DTP Peer non sono altro che stato, questo stato può variare a seconda dei sistemi dei rispettivi attori, tramite chiamata RPC o tramite risposta alla variazione di una terza proprietà.

Messaggio di identificazione

Il messaggio di identificazione è un messaggio inviato alla avvenuta connessione, ha lo scopo di notificare al *DTPHost* quale tipo di agente reale si stia connettendo, e quindi di creare l'appropriato gemello digitale.

Il suo formato è illustrato in Figura 4.5, contiene un solo parametro **identificazione**, una stringa di lunghezza variabile, la cui dimensione è la dimensione in byte del messaggi a meno di un byte.

Identification		
0	Tipo Messaggio	= 2
...	Identificazione	...

Figura 4.5: Formato del messaggio di identificazione.

4.1.1 Implementazione di DTP

Il Digital Twin Protocol nel progetto di tesi è implementato per mezzo della libreria ENet, la quale mette a disposizione socket di comunicazione orientati alla connessione, con opzionale affidabilità della trasmissione dei pacchetti, comodamente integrata all'interno di Godot: l'obiettivo del DTP è quello di sincronizzare il *real agent* con il suo *digital twin* per mezzo di **proprietà**, ovvero stato dell'agente robotico, e permettere il controllo per mezzo di **metodi**. Distinguiamo allora le entità:

- Il **DTP Peer** del **real agent**, implementato in C++ con lo scopo di essere compatibile e performante in sistemi embedded, ma ugualmente compatibile con sistemi dotati di implementazione completa dei *Berkeley socket*, come Linux; il real agent dispone di una serie di *proprietà* o attributi che rappresentano lo stato, periodicamente sincronizzate con il server, e una serie di *metodi* per il suo controllo remoto; è in grado di serializzare e deserializzare le proprietà scambiate purchè questi siano tipi *Plain Old Data* [14].
- Una istanza di Godot che implementa un **DTPNetworkManger**, ha il ruolo di configurare un *host* di ENet, accettare le connessioni, instanziare digital twin; è anche in grado di riprendere le connessioni con i *real agent* in caso di disconnessione.
- Il **DTP Peer** del **digital twin**, all'interno della medesima istanza di Godot che contiene il *DTPNetworkManger*, è responsabile della gestione degli scambi dati, come aggiornamenti di stato e chiamate RPC tra

gemello reale e gemello virtuale; implementato in GDScript, è in grado di serializzare e deserializzare alcuni tipi sfruttando la tipizzazione di GDScript.

4.1.2 Real Agent Interface

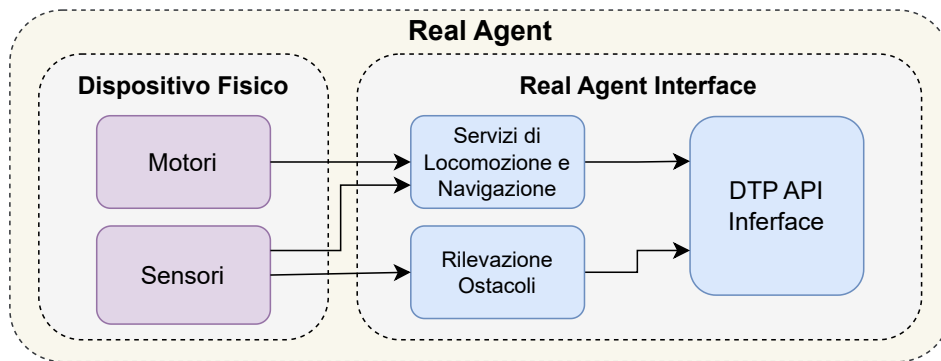


Figura 4.6: Architettura del del real agent.

Date le alte performance dei moderni microcontrollori, l’implementazione del DTP Peer per *real agent* è stata scritta in C++ 17, optando per l’uso di costrutti di alto livello e avanzati idiomi di linguaggio. Il protocollo DTP è realizzato all’interno dalla classe *BaseRAI*, una classe multiuso per la realizzazione di *Real Agent Interface*.

La classe ClientAbstraction

La **ClientAbstraction** è una classe che fornisce una interfaccia ad alto livello per la creazione di connessioni tramite ENet, usa l’idioma *Pointer to implementation "Pimpl"* [12] al fine di nascondere la sua implementazione all’utente finale e lo spazio dei nomi del linguaggio C, permette invo e ricezione messaggi, gestione di eventi di connessione e disconnessione.

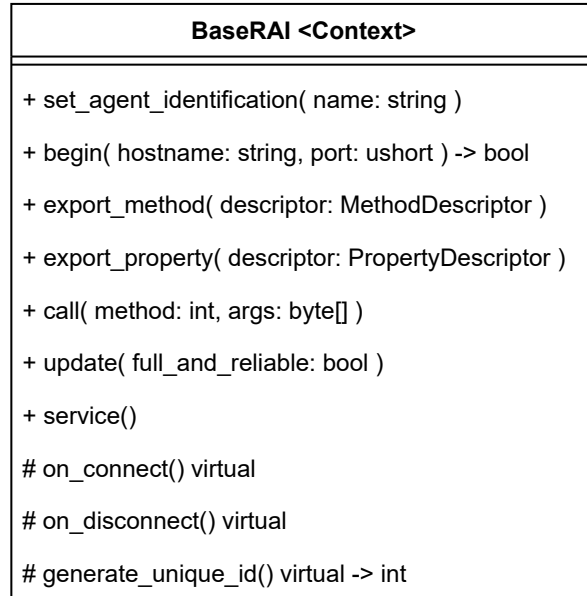


Figura 4.7: Interfaccia pubblica e protetta di BaseRAI

La classe BaseRAI

La classe template **BaseRAI** integra il *Digital Twin Protocol* sopra la *Client-Abstraction*. BaseRAI fornisce le fondamenta per l'implementazione di *Real Agent Interface* ben integrate con DTP, in particolare è possibile definire proprietà tipizzate di tipo *Plain old Data "POD"* [14] con relativi *getter* e *setter* e metodi con relativi *handler* usando una serie di strumenti dichiarativi di alto livello al fine di nascondere le complessità di DTP; usa l'idioma *Curiously Recurring Template Pattern "CRTP"* [13] allo scopo di permettere l'uso di funzioni anonime aventi come parametro la implementazione concreta di Real Agent Interface, nel particolare caso di BaseRAI, il tipo template *Context* sarà una classe che andrà ad ereditare pubblicamente BaseRAI<Context>. La Figura 4.7 mostra l'interfaccia pubblica di BaseRAI.

Di seguito una breve descrizione dei metodi offerti dalla classe BaseRAI:

- Il metodo **begin** ha lo scopo di inizializzare la socket di DTP, quindi ENet tramite in modo che si colleghi con l'istanza di Godot ospitante il *DTPNetworkManger*, che provvederà a istanziare o ricollegare un digital twin, costruire un DTP Peer di quest'ultimo e collegarlo con il DTP Peer del *real agent*; va chiamato nel setup del real agent.
- Il metodo **call** permette di effettuare chiamate a procedura remota al DTP Peer del digital twin, per mezzo di messaggi di tipo RPC.
- Il metodo **update** forza l'invio di un update, prende come argomento opzionale un booleano, di default false, che indica al metodo di inviare un aggiornamento delle proprietà completo al DTP Peer del digital twin in modo affidabile.
- Il metodo **service** esegue la logica interna di costruzione dei messaggi di aggiornamento, quindi lettura delle proprietà e loro preparazione, inoltre esegue la logica di servizio di ENet per mezzo di *ClientAbstraction*, e di conseguenza controllo dello stato di connessione, invio e ricezione pacchetti, accettazione di aggiornamento di stato e invocazioni RPC. Tale metodo deve essere chiamato periodicamente; è possibile ma non ancora implementato, eseguire tale logica insu di un thread separato, con dovuti accorgimenti sulla concorrenza.
- Il metodo **set_agent_identification** imposta il nome usato per identificare il tipo di real agent, affinché venga istanziato il corretto tipo di gemello digitale.
- Il metodo **export_method** e **export_property** sono metodi atti alla definizione dell'interfaccia pubblica DTP del robot, quindi la definizione di metodi e proprietà richiamabili dal DTP Peer del digital twin.
- I metodi virtuali protetti **on_connect** e **on_disconnect** sono utili al fine di implementare logiche rispettivamente alla connessione e alla disconnessione
- Il metodo protetto e virtuale **generate_unique_id** permette di sovrascrivere il modo con cui generare un identificativo univoco.

Esportare proprietà

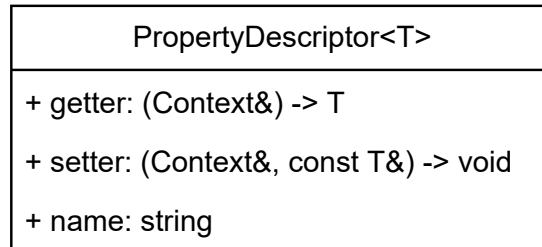


Figura 4.8: Descrittore di proprietà

Si intende per *esportare proprietà* la definizione di proprietà pubbliche e sincronizzabile tra *real twin* e *digital twin*, per mezzo di DTP, *BaseRAI* mette a disposizione il metodo `export_property` a tale scopo. Prende come input un descrittore, una struttura C++ tipizzata con il tipo della proprietà da definire, contenente due puntatori *opzionali* a funzioni, rispettivamente il *getter* e *setter*, e un nome di proprietà. I puntatori a funzione possono essere definiti per mezzo di funzioni anonime, prendono in input almeno un argomento, una referenza a *Context*; si ricorda che *Context* è esso stesso di tipo *BaseRAI* in quanto deve seguire l'idioma *C RTP*.

Di seguito un esempio usato per esportare la proprietà dello stato di carica della batteria dell'Arduino Alvik:

Codice 4.1: Esempio d'uso del metodo `export_property`

```

1  export_property<uint8_t>({
2      .name = "battery",
3      .getter = []( AlvikRAI &self ) -> uint8_t {
4          return uint8_t( self.alvik().get_battery_charge() );
5      }
6  });
```

L'attributo *name* al momento non è utilizzato poiché il DTP ancora non prevede nomi; infine l'indice corrispondente delle proprietà viene assegnato in ordine di definizione, a partire da 0.

Esportare metodi

MethodDescriptor
+ handler: (Context&, bytes[]) -> void
+ name: string

Figura 4.9: Descrittore di proprietà

Si intende per *esportare proprietà* la definizione di metodi invocabili per mezzo di DTP dall'altra istanza di DTP Peer, real agent o digital twin, *BaseRAI* mette a disposizione il metodo `export_method` a tale scopo. Prende come input un descrittore, come mostrato in Figura 4.9 gli attributi sono un *name* e un puntatore a funzione definibile anonimamente, la quale funzione ha come firma tre argomenti: Una reference a *Context*, la dimensione della payload e un puntatore ai byte della payload; si ricorda che *Context* è esso stesso di tipo *BaseRAI* in quanto deve seguire l'idioma *CRTP*.

Di seguito un esempio usato per esportare il metodo *drive*, usato per comandare in velocità il robot Arduino Alvik :

Codice 4.2: Esempio d'uso del metodo `export_method`

```

1
2     export_method({
3         .name = "drive",
4         .handler = [] (AlvikRAI &self, const void *data, int
                    size) {
5             DriveSpeed vel;
6             std::memcpy(&vel, data, sizeof(vel));
7             self.alvik().drive(vel.linear, vel.angular, CM_S,
                    RAD_S);
8         }
9     });

```

Anche per i metodi l'attributo *name* al momento non è utilizzato poiché il DTP ancora non prevede i nomi; infine l'indice corrispondente ai metodi viene assegnato in ordine di definizione, a partire da 0.

Si noti che non è garantito che la payload sia allineata in memoria, l'accesso non allineato generalmente causa eccezioni nelle architetture *RISC* come i microcontrollori ESP32, pertanto è sconsigliato eseguire il cast del puntatore *data*, e si consiglia la copia sullo stack via *memcpy*.

4.1.3 DTPNetwork e DTPPeer in Godot

La implementazione in Godot del DTP è scritta in GDScript e usa la implementazione di ENet integrata in Godot stesso, perfettamente compatibile con la release ufficiale usata con la implementazione per real agent. Poiché l'istanza di Godot si comporta da server per ENet, è stato necessario produrre due differenti oggetti, un oggetto che si occupi della gestione delle connessioni e la istanziatura dei Digital Twin, il **DTPNetwork**, una seconda classe che si occupi di gestire gli eventi per la singola istanza del gemello virtuale, il **DTPPeer**.

Inoltre, allo scopo di facilitare l'integrazione con l'implementazione per real agent e lo sviluppo in Godot del framework, un sistema di serializzazione e deserializzazione di tipi base di Godot compatibile con quanto implementato per real agent è stato costruito.

DTPNetwork

L'oggetto **DTPNetwork** è un *nodo* in Godot responsabile della creazione e gestione dell'infrastruttura che collega *real agent* e *digital twin*, in particolare dell'istanziatura di gemelli digitali; nel suo inspector sono presenti le seguenti proprietà:

- Il dizionario **Digital Twins**, associa il codice di *identificazione* di un *real agent* con la *PackedScene* del gemello digitale; la configurazione di questa tabella permette di gestire gemelli digitali di agenti robotici differenti.

- Due campi, **Host address** e un **Host port**, rispettivamente l'indirizzo di rete e la porta su cui il server ascolta una volta inizializzato.
- Una campo **Spawn path**, che deve contenere il percorso del nodo contenitore su cui istanziare i gemelli digitali.

Alla inizializzazione, DTPNetwork tenterà di istanziare e inizializzare una *ENetConnection* su indirizzo e porta specificati nella configurazione, qualora vi fosse un errore, scriverà nel log di sistema di Godot tale evento e ripristinerà lo stato precedente.

Una volta inizializzato con successo, DTPNetwork è in grado di processare gli eventi, quindi accettare connessioni, istanziare gemelli digitali e liberare risorse occupate da peer sconnessi; tale logica è implementata all'interno del metodo privato *_service*, richiamato a sua volta dal metodo di life-cycle *_process* di Godot.

La connessione di un real agent può avvenire in due contesti diversi a seconda se si tratta di un recupero da disconnessione o meno:

1. Accetta la connessione di ENet. Nel caso in cui esista già un gemello digitale corrispondente al *peer_id* della nuova chiamata, vorrà dire che si tratta di una riconnessione, allora viene eliminato il gemello digitale disconnesso e si procede allo step 4.
2. Processerà in messaggi del nuovo peer finché non riceverà un pacchetto di identificazione.
3. Verifica che esiste un modello di gemello digitale compatibile con l'identificazione del real agent, in caso contrario chiuderà la connessione con questo e segnerà un messaggio di *warning* nel log di sistema di Godot.
4. Viene istanziato il digital twin corrispondente, inizializzato il DTPPeer del gemello digitale, aggiunto questo nell'albero di scena come figlio del nodo indicato da *spawn_path*, e infine invocato il metodo RPC *spawn_robots* della classe *MultiplayerNetwork* al fine di notificare le altre possibili istanze di Godot connesse del nuovo gemello digitale.

DTPPeer

Una volta istanziato il gemello digitale, l'oggetto di tipo nodo DTPPeer è delegato alla comunicazione da e verso il DTPPeer del real agent. Le sue responsabilità sono:

- Invocare chiamate a procedura remota sul real agent.
- Accettare ed eseguire chiamate a procedura remota dal real agent.
- Inviare aggiornamenti di stato.
- Elaborare i messaggi ricevuti dal DTP
- Interpretare i messaggi di aggiornamento, e opzionalmente, applicarli sul *ghost*.

Al fine di offrire una implementazione flessibile del DMDTE, è stata sviluppata una soluzione per la codifica e decodifica di valori Godot, tale set di strumenti permette di serializzare e deserializzare: numeri interi da 8 bit a 32 bit, numeri reali, vettori a due o tre dimensioni interi e reali, stringhe, array di bytes, array di interi a 32 bit e array di reali; la serializzazione avviene automaticamente convertendo i tipi nel loro corrispettivo tipo C (interi con interi, reali con reali), allo stesso modo la decodifica converte array di bytes nel loro corrispettivo tipo Godot.

Tale lavoro sui tipi ha permesso la produzione di un sistema interattivo da interfaccia utente per la definizione di proprietà scambiate per DTP e l'invocazione delle chiamate RPC, entrambi le proprietà corredate di tipizzazione.

Come nel caso dei descrittori di *BaseRAI*, abbiamo relativamente per la definizione della proprietà un descrittore contenente i seguenti attributi:

- Un **indice** che identifica la proprietà DTP, di tipo *uint8*.
- Un **nome** da assegnare alla proprietà, questo può opzionalmente corrispondere con il nome dell'attributo del ghost relativo a tale stato, al fine di aggiornare automaticamente questo.

- Un **tipo** serializzabile.

Analogamente, il descrittore dei metodi contiene le seguenti proprietà:

- Un **indice** che identifica il metodo DTP.
- Un **nome** alias di tale metodo, da usare per l'invocazione; nel caso il ghost sia dotato di un metodo con il medesimo nome, le invocazioni RPC da parte del real agent invocheranno tale metodo.
- Una lista di **tipi** corrispondenti ai parametri da inoltrare con la invocazione.

Come già discusso nella descrizione del DMDTE, il *ghost* deve replicare il real agent all'interno dell'ambiente virtuale, il DTPPeer infatti tende ad assegnare gli aggiornamenti di stato delle proprietà al ghost del digital twin e delegare invocazione di metodi: pertanto, seppure non necessario, la soluzione proposta impiega il ghost come interfaccia al real agent.

Elaborazione dei messaggi DTP

L'elaborazione dei messaggi DTP avviene solo se è attiva una connessione con il real agent, in tale caso viene eseguito il metodo privato `_service` il quale decodifica il pacchetto ed esegue il dispaccio dell'evento al relativo handler. Al fine di evitare aggiornamenti continui, il quale può causare congestione nella rete e sovraccarichi, un sistema di aggiornamento a intervalli è implementato e gestibile dall'utente, in tale intervallo aggiornamenti e chiamate RPC vengono immagazzinate in apposite cache, alla fine di tale intervallo tali cache vengono svuotate inviando i messaggi all'agente robotico.

L'elaborazione dei messaggi di aggiornamento avviene per interpretazione congiunta del messaggio DTP e del descrittore della proprietà, dall'elaborazione del messaggio viene costruito un dizionario il quale verrà immagazzinato nello stato interno del DTPPeer. Nel caso cui sia associato un ghost, allora il DTPPeer proverà ad assegnare le proprietà ricevute al ghost.

L'elaborazione delle chiamate RPC avviene solo se è presente il ghost, questo allora deve possedere un metodo con il nome alias definito nel de-

scrittore, quindi procederà alla decodifica degli argomenti e l'invocazione del metodo.

4.2 Digital Twin

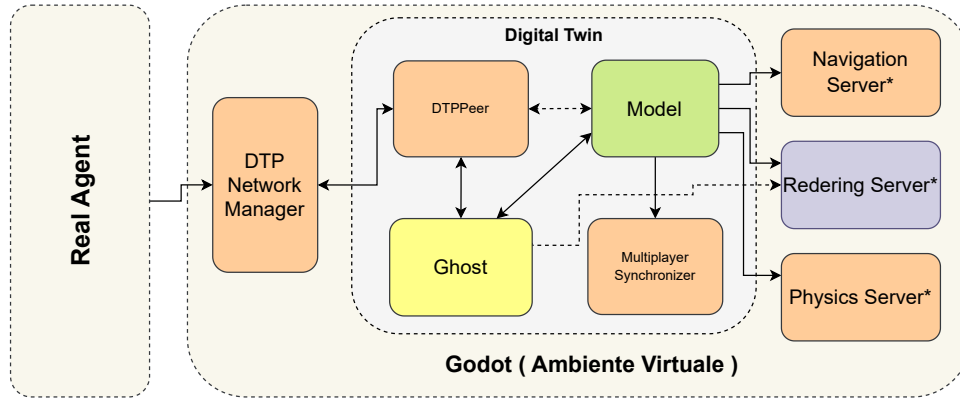


Figura 4.10: Architettura del digital twin.

I Digital Twin vengono rappresentati all'interno di Godot come delle scene. La loro radice è di tipo *nodo*, un contenitore generico inseribile all'interno della scena di Godot, a cui vi è allegato lo script *DigitalTwin*, cui svolge ruolo di *facade*, espone e raccoglie le componenti che formano il gemello digitale stesso secondo il framework DMDTE: **model**, **ghost** e **DTPPeer**.

I gemelli digitali possono essere creati in due modi diversi:

- Tramite **DTP** verrà creato il corrispettivo digitale e ad esso assegnato un *DTPPeer*, allora il gemello digitale è **locale**.
- Tramite **MultiplayerAPI**, il gemello digitale di una istanza di Godot remota viene automaticamente creato da *MultiplayerSpawner*, o *MultiplayerNetwork* nel caso il gemello digitale sia stato appena creato nella istanza remota, tale gemello digitale allora sarà **remoto**, non controllabile, transparentemente sincronizzato per mezzo del *MultiplayerSynchronizer*.

Model

Il model è una oggetto all'interno della scena, questo oggetto può essere capace di navigare e interagire con ambiente virtuale e pertanto controllare questo oggetto implica controllare il gemello digitale.

Il model deve emulare il più fedelmente possibile il comportamento del gemello reale, affinché gli stessi input siano replicabili: per esempio il model dell'Arduino Alvik prodotto in fase di sperimentazione presenta la medesima funzione **drive** del corrispondente gemello reale.

Si precisa che nel caso il gemello digitale sia *remoto*, queste operazioni possono essere eseguite solo dalla istanza autorevole.

Ghost

Il ghost è un oggetto passivo che ha il solo scopo di contenere lo stato e rappresentare il gemello reale del digital twin. È fondamentale che il ghost abbia le stesse proprietà del gemello reale, al fine di permettere il DTPPeer di poterne aggiornare in modo trasparente lo stato.

Può opzionalmente essere dotato di un modello tridimensionale, ma non deve essere dotato di fisica e collisioni.

MultiplayerSynchronizer

Il **MultiplayerSynchronizer** è un nodo di Godot responsabile della sincronizzazione delle entità tra istanze collegate tramite la **MultiplayerAPI** di alto livello. Tale strumento viene configurato tramite assegnazione di percorsi a proprietà di nodi della scena. È necessario dunque indicare nel *synchronizer* del gemello digitale quelli attributi che devono essere condivisi con le altre istanze remote di Godot; di particolare interesse nel contesto di agenti robotici è la sincronizzazione della posa del *model*, in particolare dei suoi attributi *posizione* e *rotazione*.

Controller

I controller sono degli script che permettono di definire o estendere determinati comportamenti a un digital twin: tramite la manipolazione del solo *model*, è possibile permettere agli agenti la navigazione dell'ambiente virtuale tramite input utente oppure i servizi di navigazione e path-finding integrati in Godot; oppure ancora è possibile definire comportamenti non strettamente legati al controllo del robot, come successivamente viene mostrato per la

demo d'uso, i controller verranno usati per implementare le logiche di un videogioco. I controller vanno inseriti in nodi all'interno dell'albero del gemello digitale.

Eventuale stato detenuto dai controller può essere sincronizzato con le altre istanze di Godot connesse per mezzo del *MultiplayerSynchronizer*.

4.3 Replicazione scena

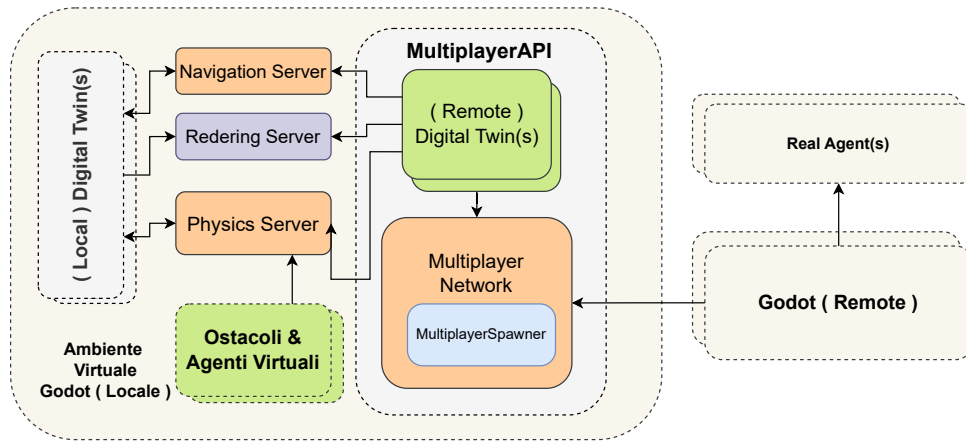


Figura 4.11: Architettura della MultiplayerAPI e la sua interazioni con le componenti.

La MultiplayerAPI di Godot automatizza gran parte del lavoro di interconnessione e sincronizzazione tra le istanze.

MultiplayerNetwork

Al fine di rendere operativa la componente di rete è stata sufficiente l'implementazione della classe *MultiplayerNetwork* per la inizializzazione di una particolare implementazione di *ENetPacketPeer*, una interfaccia che astrae l'implementazione di un protocollo di trasporto alla stack dedicato al multiplayer.

La classe *MultiplayerNetwork* espone all'utente finale i seguenti metodi:

- Il metodo **create_server**, per la creazione di una istanza server della MultiplayerAPI. Prende come unico parametro il numero di porta di rete dove porre il server in ascolto.
- Il metodo **create_client**, per la creazione di una istanza client della MultiplayerAPI. Prende due parametri: indirizzo di rete e numero porta del server cui connettersi.

Una volta che una istanza client entra in connessione con un server, questi scambieranno le istanze di gemelli digitali collegati tramite DTP in modo del tutto trasparente all'utente.

Replicazione della scena

Il nodo di Godot *MultiplayerSpawner* si occupa della replicazione della scena nelle nuove istanze connesse, prende come configurazione un set di *scene* istanziabili, nel caso del framework DMDTE, le scene che incapsulano i gemelli digitali: il nodo si occuperà della replicazione della scena dell'istanza server alle nuove istanze client collegate.

Capitolo 5

Sperimentazione

Gran parte del lavoro svolto nella fase di implementazione è stato fatto in stretta corrispondenza con lavori di sperimentazione, tramite tale processo è stato possibile infatti correggere errori, fissare nuovi obiettivi e implementare nuove funzionalità.

Come obiettivo di sperimentazione è stato prefissato la realizzazione di un videogioco di combattimento tra carri con tecnologia digital twin.

5.1 Integrazione di Arduino Alvik

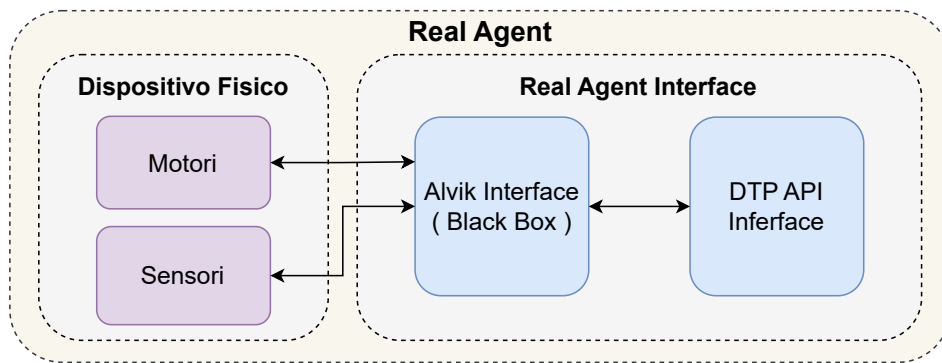


Figura 5.1: Architettura del gemello reale di Arduino Alvik.

Real Agent Interface

Come ampiamente trattato nel capitolo precedente, una concreta implementazione della classe *BaseRAI* fornisce tutti gli strumenti necessari e di alto livello per fornire una interfaccia compatibile DTP, con la chicca di poter tipizzare le proprietà.

AlvikRAI (DTP)
+ battery: int + is_battery_charging: bool + pose: float[3] + drive_speed: float[2]
+ drive(linear: float, angular: float) + reset_pose(x: float, y: float, theta: float)

Figura 5.2: Interfaccia DTP del robot Arduino Alvik.

Per il progetto di tesi è stata quindi realizzata una *Real Agent Interface* per l'agente robotico mobile Arduino Alvik, completa di tutte le comodità necessarie al comando del sistema di locomozione dell'agente e il monitoraggio di stato, come velocità, posa, stato di carica della batteria. In Figura 5.2 la interfaccia esposta API per mezzo di DTP.

Tutti i sottosistemi dell'agente robotico, come locomozione, odometria, stato della batteria e sensori, sono esposti da una interfaccia API open source sviluppata da Arduino stessa [15], pertanto consideriamo tale interfaccia una *scatola nera* o *black box* ai fini del lavoro svolto.

Di seguito una descrizione delle proprietà e metodi esposti su DTP:

- La proprietà **battery** indica lo stato di carica della batteria dell'agente, è un intero con valori da 0 a 100.

- La proprietà **is_battery_charging** indica se la batteria dell'agente è in ricarica, è di tipo booleano.
- La proprietà **pose** espone la posa del robot come array di 3 numeri *float*, questi tre numeri devono essere interpretati come: un vettore a due dimensioni che rappresenta la posizione nel piano del robot in *centimetri* di coordinate x e y ; il terzo numero è un reale, con il nome in letteratura di *theta*, rappresenta la direzione o angolo nel piano in *radianti*.
- La proprietà **drive_speed** espone un array di due numeri reali cui le singole componenti rappresentano rispettivamente *velocità lineare* (cm/s) e *velocità angolare* (rad/s) dell'agente.
- Il metodo **drive** prende come parametri una *velocità lineare* (cm/s) e una *velocità angolare* (rad/s) obiettivo, l'agente allora proverà a muoversi a tali velocità: questa è l'unica funzione necessaria e sufficiente per la manipolazione spaziale del robot.
- Il metodo **reset_pose** prende in input tre numeri reali x , y e $theta$ di tipo *float* che rappresentano una posa nello spazio, con lo scopo di ripristinare o sostituire la posa attuale con quella nuova.

Non è necessaria una analisi approfondita di quanto esposto, poiché l'interfaccia espone direttamente funzionalità fornite dalle API di Arduino.

Gemello digitale

Il **gemello digitale** invece, è dotato di modello tridimensionale dalla forma di un carro armato in stile cartonesco dotato di cingolati animati, forma e presenza fisica, ed emula un comportamento di movimento simile alla sua controparte reale.

Il suo model si muove per mezzo della funzione **drive**, analoga alla funzione del gemello reale, imposta velocità lineare e velocità angolare obiettivi all'interno dello stato dell'agente. Successivamente, durante l'elaborazione della fisica, il model proverà a muoversi nell'ambiente virtuale, in caso di

collisioni con oggetti allora questo fermerà il suo moto lungo la specifica direzione e avrà allora velocità nulla.

Una volta calcolata la velocità finale, dopo l'elaborazione della fisica, viene estratto il vettore tridimensionale che ne rappresenta il moto, decomposto nuovamente in velocità lineare e angolare, e infine invocata una chiamata a procedura remota al corrispondente metodo *drive* del gemello reale per mezzo di DTP, usando come parametri le velocità finali precedentemente calcolate.

Infine, la posa del model viene forzatamente posta in vicinanza al ghost usando interpolazione lineare.

Ulteriore software prodotto

A supporto del lavoro svolto sul dispositivo embedded, sono state sviluppate una serie di utilità per semplificare la qualità di vita, comodamente disponibili all'interno di una interfaccia C++ e ordinate in rispettivi spazi dei nomi.

Funzione per la gestione dei LED

La scheda a microcontrollore usata come piattaforma computazione dall'Arduino Alvik è un **Arduino ESP32 Nano BLE** dotato di tre LED, di cui due manipolabili tramite GPIO. Uno spazio dei nomi dedicato ai LED è stato prodotto, dotato di una procedura di inizializzazione *init* e una serie di procedure per la manipolazione dei LED:

- Un LED **RGB** manipolabile in logica di pulldown, sono state scritte funzioni per accendere o spegnere i tre colori **rosso**, **verde** e **blu** in modo non equivocabile.
- Un LED di colore arancione chiamato **builtin** in manipolabile in logica di pullup.

Tali funzionalità di aiuto sono state prodotte al fine di rendere non ambigua e pulita la manipolazione dei LED, indispensabili come mezzo visivo da parte dell'utente per la percezione dello stato del dispositivo.

Preferenze e stack di rete

È stato sviluppando un sistema per la gestione e configurazione di credenziali della rete wireless e coordinate dell'host Godot ospitante l'ambiente virtuale e digital twin, così da non essere necessaria la scrittura di tali dati all'interno del sorgente e rendere più flessibile la riconfigurazione del dispositivo.

Tale sistema prevede la creazione di un task dedicato in ascolto sulla seriale del dispositivo, in attesa di una specifica payload JSON, una volta verificato che tale payload contiene la configurazione richiesta (SSID, password, indirizzo e porta host), la configurazione viene salvata nella memoria flash dell'ESP32 e quest'ultimo riavviato.

Sono state dedicate per il sistema di configurazione e stack di rete le funzioni: `preferences::init` per inizializzare il sistema e creare il task in ascolto, `preferences::get_preferences` per interrogare le preferenze, `wifi::init` per inizializzare lo stack di rete e quindi la connessione alla rete wireless.

5.2 Port di ENet per ESP32

Il sistema operativo real-time *FreeRTOS* per sistemi embedded non fornisce un classico ambiente di esecuzione come Windows e Linux, pertanto la libreria ENet non risulta direttamente compatibile con tale sistema.

Data l'importanza strategica di ENet ai fini del progetto, dopo una attenta analisi del sorgente e della documentazione di lwIP, lo stack di rete TCP/IP implementato in FreeRTOS, sono stati individuati una serie di parametri non compatibili usati nelle funzioni di manipolazione delle socket:

- La funzione **getnameinfo** per la risoluzione del nome di rete di un indirizzo IP in lwIP non implementa il *flag* `NI_NAMEREQD`, tale flag obbliga la funzione a risolvere il nome di rete e in caso di fallimento restituire un errore, è stato sufficiente rimuovere l'uso di questo flag.
- Due funzioni, **recvmsg** e **sendmsg**, servono rispettivamente per accogliere e inviare messaggi tramite la socket, queste funzioni possono

accettare il parametro di flag `MSG_NOSIGNAL` in sistemi UNIX, tale parametro istruisce il sistema operativo di non inviare un segnale di arresto di tipo `SIGPIPE` e quindi mandare in crash l'applicativo nel caso di *disconnessione improvvisa della socket*, permettendo a quest'ultimo la gestione della condizione di errore. Il sistema operativo embedded usato dalla piattaforma ESP32, FreeRTOS, non implementa i segnali, è stato sufficiente ancora una volta rimuovere l'uso di questo flag in entrambe le invocazioni.

Da tale analisi e correzioni è stato prodotto un fork compatibile e liberamente disponibile [16]. Si rassicura il lettore che le correzioni eseguite sono cautamente applicate se e solo se la piattaforma target di compilazione è un ESP32, usando le dovute direttive condizionali.

5.3 Prove sperimentali

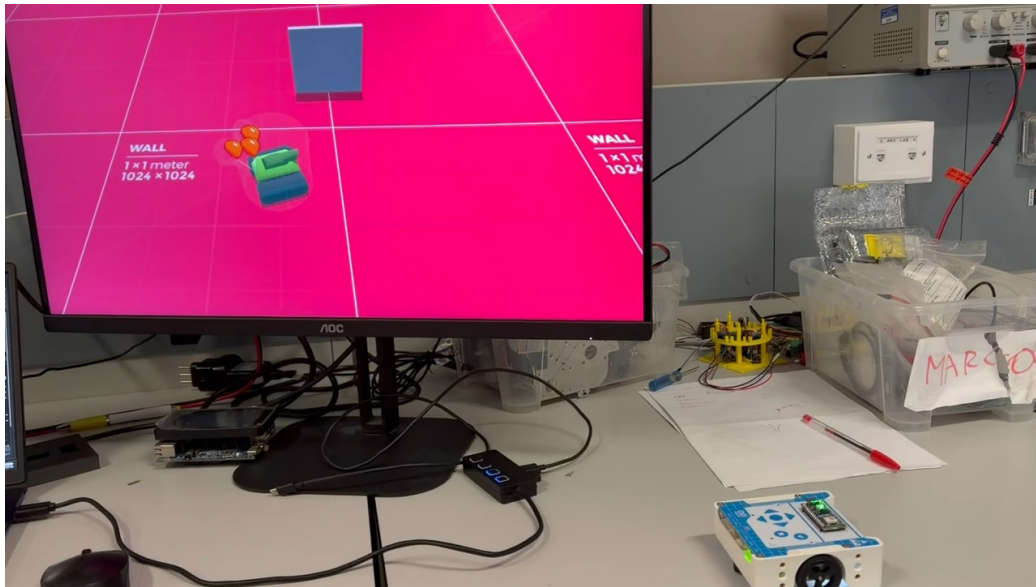


Figura 5.3: Gemello digitale singolo agente con guida a sterzata.

Singolo agente

Le prime prove sperimentali hanno riguardato il controllo di un singolo gemello digitale. In questa fase l'autore ha costruito familiarità con l'agente robotico e la cinematica differenziale; sono stati esplorati sistemi di controllo di guida differenti quali un *controllo a sterzata* e un *controllo polare* per la guida del robot.

Infine, è stato costruito un ambiente virtuale per la navigazione, composto da un piano e un ostacolo, come mostrato in Figura 5.3. In tale ambiente è stato verificato che, qualora il gemello digitale collidesse con un muro e arrestasse il suo moto, di conseguenza anche il gemello reale arrestasse il suo moto.

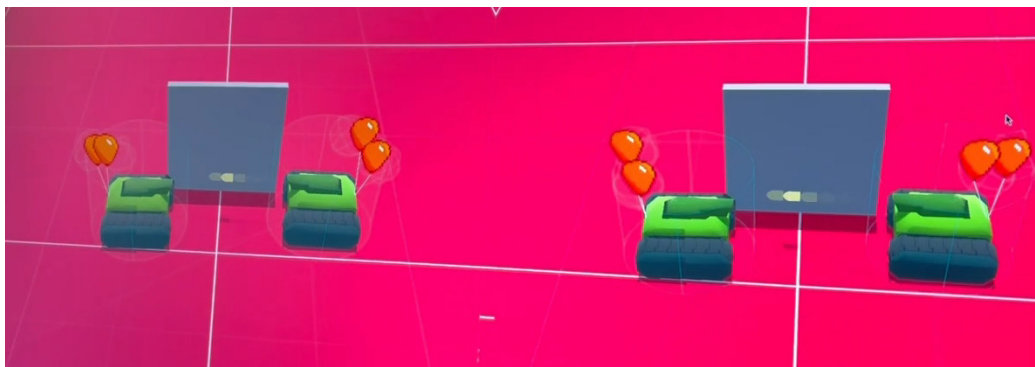


Figura 5.4: Due istanze di Godot che espongono l'una all'altra un gemello digitale, implementati anche alcune feature di videogioco.

Multi-agente distribuito

Concluse le prove singolo-agente, è stata brevemente attestata la capacità del sistema software di gestire più agenti in una singola istanza.

Posto l'obiettivo di rendere distribuito il software, è stato svolto il lavoro necessario affinché si potessero mettere in comunicazione più istanze di Godot in grado di esporre i gemelli digitali ad essa collegata: in tale fase sono state affrontate e superate con successo le eventuali complicazioni dovuta all'architettura della *MultiplayerAPI*, in particolare la gestione della *authority*, la istanziamento degli agenti remoti e la loro sincronizzazione.

In Figura 5.4 sono mostrate due istanze separate di Godot che condividono ambiente virtuale e gemelli digitali.

Multi-agente distribuito e eterogeneo



Figura 5.5: Aggiunta di un terzo agente di tipo semaforo.

Durante la stesura delle fasi conclusive dell’elaborato, è stato osservato che il progetto finale potesse arricchirsi ulteriormente permettendo il collegamento di tipi diversi di gemelli digitali.

Una volta implementata la logica all’interno del sistema, questa è stata corretta da eventuali errori e provata.

Come mostrato in Figura 5.5, è stato implementato un gemello digitale di un oggetto semaforo. Una relativa *Real Agent Interface* è stata prodotta per tale oggetto ed eseguita da una board personalizzata dotata di microcontrollore Arduino Nano ESP32 BLE, il medesimo dell’Arduino Alvik, e di pulsantiera.

Realizzazione di un videogioco digital twin

Come obiettivo conclusivo del percorso sperimentale, è stato prodotto un prototipo di videogioco di combattimento tra carri armati usando la tecnologia digital twin implementata.

Con l'obiettivo di supportare le dinamiche di gioco sulla infrastruttura realizzata, i gemelli digitali originali degli agenti Alvik sono stati estesi con le seguenti componenti e aggiunte:

- È stato implementato un **sistema di vita** sul gemello digitale del carro. Ogni carro dispone di *tre unità di vita*, raffigurate visivamente da altrettanti *palloncini* legati al retro del carro. Ogni qualvolta il carro riceve un danno perde una unità di vita e un palloncino si scollega e vola via, inoltre: se il carro ha terminato le vite, il carro viene arrestato e viene mostrata visivamente una bandiera bianca a simboleggiarne la resa; altrimenti nel caso avesse ancora vite disponibili otterrà temporaneamente l'invincibilità e non potrà essere colpito da altri giocatori.
- A supporto del sistema precedente, è stato implementato un **sistema di shooting** dotato di tempo di ricarica. È stato prodotto un oggetto **bullet** (proiettile) e relativo assets, in grado di muoversi lungo un moto parabolico, collidere con oggetti e dotato di tempo di vita e massima distanza. Qualora in bullet interagisca con un carro avvierà le opportune routine di danno.
- Un sistema di ripristino è stato implementato affinché lo stato di vita dei carri possa essere ripristinato.

Al fine di rendere l'esperienza tanto più coinvolgente possibile, sono stati prodotti assets originali e personalizzati assets pubblici e gratuiti, inoltre è stata realizzata una opportuna arena di gioco.

Conclusione

L'implementazione di *Distributed Multi-Agent Digital Twin Environment* prodotta come progetto di tesi offre una vasta gamma di strumenti atti a facilitare lo sviluppo di soluzioni digital twin multi agente con il motore di gioco Godot.

È stato concretamente realizzato il protocollo *Digital Twin Protocol*, fondamentale di tutto il progetto, il quale non solo permette uno scambio bidirezionale semplice e veloce di stato tra i due gemelli, ma anche invocazioni a procedura remota.

Al supporto di DTP è stata realizzata una libreria semplice e immediata per la definizione di interfacce *Real Agent Interface* in C++, enfatizzando l'uso di più semplici ma altrettanto sofisticati paradigmi dichiarativi di alto livello, con particolare attenzione alla facilità d'uso e la portabilità per dispositivi embedded e IoT, a cui è seguita una digressione di percorso atta a rendere compatibile la libreria ENet con sistemi embedded basati su ESP32.

Infine, una suite di componenti e strumenti sono stati realizzati per il motore di gioco Godot, atti alla realizzazione di complessi sistemi multi-agente di gemelli digitali, la quale sfrutta pienamente le API offerte dalla piattaforma per il networking, semplificando drasticamente la realizzazione di sistemi digital twin distribuiti.

I risultati ottenuti soddisfano pienamente gli obiettivi implementativi prefissati, confermando la solidità dell'architettura proposta nell'articolo scientifico su cui si fonda[1].

La conclusione di questo lavoro non è un punto di arrivo definitivo, bensì pone le basi per ulteriori e interessanti sviluppi.

In primo luogo, con l'obiettivo di migliorare la qualità del lavoro svolto finora con il protocollo DTP, si intende introdurre un formato di serializzazione dati binario e leggero, così da ridurre ambiguità e potenziali errori.

Di notevole interesse per lavori futuri è la ricerca e implementazione di una soluzione peer-to-peer per la comunicazione tra istanze del framework, così da eliminare la corrente soluzione di natura prettamente client-server, e quindi estendere ulteriormente il framework introducendo robusti sistemi ridondanti e distribuiti di gemelli digitali multi agente.

Infine, sarebbe interessante esplorare la portabilità del framework, o la realizzazione di una sua particolare implementazione compatibile, in grado di eseguire in sistemi embedded.

Bibliografia

- [1] Miriana Russo, Corrado Santoro, Federico Fausto Santoro, and Alessio Tudisco. Advancing robotic systems with distributed multi-agent digital twins: A scalable and adaptive framework. *SN Computer Science*, 6(8): 975, Nov 2025. ISSN 2661-8907. doi: 10.1007/s42979-025-04525-w. URL <https://doi.org/10.1007/s42979-025-04525-w>.
- [2] Zepeng Cai. Application and development of digital twins in smart cities. In *2024 4th International Conference on Big Data Engineering and Education (BDEE)*, pages 27–31, 2024. doi: 10.1109/BDEE63226.2024.00012.
- [3] Cor Verdouw, Bedir Tekinerdogan, Adrie Beulens, and Sjaak Wolfert. Digital twins in smart farming. *Agricultural Systems*, 189:103046, 2021. ISSN 0308-521X. doi: <https://doi.org/10.1016/j.agry.2020.103046>. URL <https://www.sciencedirect.com/science/article/pii/S0308521X20309070>.
- [4] Nabil Mchirgui, Nordine Quadar, Habib Kraiem, and Ahmed Lakhssassi. The applications and challenges of digital twin technology in smart grids: A comprehensive review. *Applied Sciences*, 14:10933, 11 2024. doi: 10.3390/app142310933.
- [5] Godot Engine Community. Godot engine 4: Free and open source 2d and 3d game engine, 2024. URL <https://godotengine.org>.
- [6] Lee Salzman. Enet: Reliable udp networking library, 2024. URL <http://enet.bespin.org/>. C library for reliable UDP communication.

- [7] Jorrit Rouwe. Jolt physics: A multi core friendly rigid body physics and collision detection library, 2024. URL <https://github.com/jrouwe/JoltPhysics>. GitHub Repository.
- [8] Godot Engine Community. Gdscript basics – godot engine (stable) documentation in english, 2025. URL https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript_basics.html.
- [9] Godot Engine Community. High-level multiplayer – godot engine (stable) documentation in english, 2025. URL https://docs.godotengine.org/en/stable/tutorials/networking/high_level_multiplayer.html.
- [10] Arduino. Arduino alvik: The micropython-powered robot for education, 2025. URL <https://www.arduino.cc/education/arduino-alvik/>.
- [11] Blender Foundation. Blender: Free and open source 3d creation suite, 2025. URL <https://www.blender.org>.
- [12] cppreference.com contributors. Pimpl idiom. Online Reference, 2025. URL <https://en.cppreference.com/w/cpp/language/pimpl.html>.
- [13] cppreference.com contributors. Curiously recurring template pattern (CRTP). Online Reference, 2025. URL <https://en.cppreference.com/w/cpp/language/crtp.html>.
- [14] Brown, Walter E. Pod: Plain ol’ data. C++ Language Note. Fermilab (Fermi National Accelerator Laboratory), 1999. URL <https://web.archive.org/web/20161203130543/http://www.fnal.gov/docs/working-groups/fpcltf/Pkg/ISOcxx/doc/POD.html>. Ultimo aggiornamento: 29 Novembre 1999.
- [15] Arduino. Arduino alvik library. https://github.com/arduino-libraries/Arduino_Alvik, 2025. GitHub repository.

- [16] marcospampi. Enet for esp32, 2025. URL <https://github.com/marcospampi/enet>.